

HLD Case Study 10: Microblogging Platform (Twitter / X)

Key Focus Areas: Massive fan-out write amplification, hot partition elimination, the Celebrity / Influencer problem, Twitter Snowflake 64-bit ID generation, and timeline caching.

1. Problem Statement & Functional Requirements

Design a microblogging platform where users publish short tweets, follow accounts, and consume a real-time home timeline.

Requirements:

1. Post tweets (< 280 characters, optional media).
 2. Follow and unfollow users.
 3. High-speed home timeline generation ($< 50\text{ms}$ latency).
 4. Scale to 400 million DAU, 600 million tweets/day, and 6 billion timeline views/day.
-

2. High-Level Architecture Diagram

flowchart TD

User["User Client"] → CDN["CDN (Images & Static Assets)"]

User → ALB["Application Load Balancer"]

ALB → Gateway["API Gateway"]

Gateway → TweetIngest["Tweet Post Service"]

TweetIngest → Snowflake["Snowflake ID Generator (64-bit Time-Sorted IDs)"]

TweetIngest → TweetDB["Tweet Store (Cassandra / CockroachDB)"]

TweetIngest → Kafka["Kafka: tweet_created_events"]

Kafka → FanoutCluster["Timeline Fan-Out Workers"]

FanoutCluster → SocialGraph["Social Graph Service (FlockDB / Neo4j)"]

FanoutCluster → TimelineCache["Redis Timeline Cluster (User Timelines)"]

Gateway → TimelineSvc["Timeline Query Service"]

TimelineSvc → TimelineCache

TimelineSvc → TweetDB

3. The 64-Bit Snowflake ID Generation Algorithm

Standard auto-incrementing database IDs do not work across sharded databases, and UUIDv4 strings are 128 bits, non-indexable, and unordered. Twitter created **Snowflake**: a 64-bit globally unique, time-ordered integer:

flowchart LR

Bit1["1 Bit
(Unused Sign Bit: 0)"] --- Bit2["41 Bits
(Epoch Timestamp in ms)"]

Bit2 --- Bit3["10 Bits
(Machine / Datacenter Node ID: 1024 nodes)"]

Bit3 --- Bit4["12 Bits
(Local Sequence Counter: 4096 IDs per ms per node)"]

Why Snowflake is Superior:

1. **Naturally Time-Ordered:** Sorting by Snowflake ID is identical to sorting by creation time.
2. **Decentralized:** Independent machines generate unique IDs without inter-server network coordination!

3. **High Throughput:** Each node can generate \$4,096,000 \text{ IDs per second}\$.

4. Solving the Celebrity / Hot Partition Problem

```

flowchart TD
    subgraph Standard_Fanout ["Normal User Posts (e.g. 300 Followers)"]
        NormalPost["User posts tweet"] --> WritePush["Fanout Worker appends tweet_id into"]
    end

    subgraph Celebrity_Fanout ["Celebrity Posts (e.g. 100 Million Followers)"]
        CelebPost["Celebrity posts tweet"] --> BlockFanout["DO NOT FAN OUT!<br/>(Pushing"]
        BlockFanout --> CelebTimeline["Store tweet only in Celebrity's personal timeline"]
    end

    FollowerOpensApp["Follower opens home timeline"] --> Merge["Fetch user's cached t"]
  
```

This hybrid push-pull design eliminates write amplification while preserving sub-50ms read latency across the entire platform.